



JS

JS Aula 02.1:

Arrays e Objetos

O que são arrays em JavaScript?

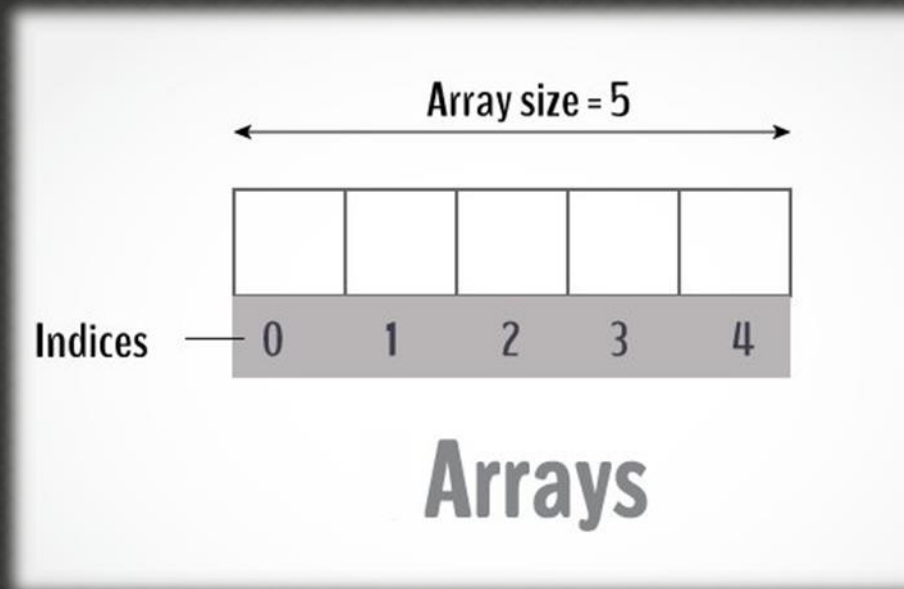


O **array** é um **objeto** no JavaScript, ele nos auxilia pois podemos utilizar uma única variável para armazenar uma **lista** de diferentes elementos.

Por exemplo, imagine que seja necessário armazenar os nomes de todos os alunos de uma sala de aula.

Criar uma **variável** para cada aluno seria, no mínimo, trabalhoso, dependendo do tamanho dos dados eu diria inviável.

Nesta ideia podemos então criar a variável **alunos** e com isso armazenar o nome de **todos os alunos**.



JS

Como declarar um array no JavaScript?



Para utilizar os Arrays precisamos declarar uma variável. E ao atribuir um valor, vamos colocar entre `[ ]` e separar cada um por `,`. Por exemplo:

```
let Produtos = ['Computador', 'Notebook', 'Celular', 'Tablet']
```

Nós podemos declarar arrays também sem nenhum item e tamanho.

```
let Produtos = []
```

JS

Exibindo os dados de um Array



```
let Produtos = ['Computador', 'Notebook', 'Celular', 'Tablet']
```

Para acessar um item de um array é muito simples, basta utilizarmos o **índice** da posição do item.

```
console.log(Produtos[0])
```

Nesse caso será exibido o valor “**Computador**” que é o primeiro item do nosso array.

JS

Percorrendo um Array com forEach



```
let Produtos = ['Computador', 'Notebook', 'Celular', 'Tablet']
```

Em alguns casos pode ser necessário percorrer uma determinada lista e exibir seus valores de forma automática em um loop de repetição. Nesse caso podemos utilizar a estrutura `forEach`, como no exemplo a seguir:

```
Produtos.forEach(produto => {  
    console.log(produto)  
})
```

Nesse caso será exibido todos os itens da lista:

Computador
Notebook
Celular
Tablet

JS

Objetos no JavaScript



Nós utilizamos **objetos** para agrupar valores que possuem **propriedades** e **funções**, podemos por exemplo fazer uma analogia aos objetos do mundo real. Um carro que possui **cor** e **marca**, ele também realiza **ações** como **acelerar** e **frear**.

Para declarar um objeto a sintaxe é bem parecida a que utilizamos para declarar **arrays**, porém trocamos os `[ ]` por `{ }`. Onde as chaves, valores e funções são declaradas entre `{ }`, conforme abaixo:

```
let carro = {  
  modelo: 'gol',  
  cor: 'vermelho',  
  acelerar(){  
    console.log('Acelerando...')  
  },  
  frear(){  
    console.log('Freando')  
  },  
}
```

JS

Como acessar as propriedades ou funções de um Objeto?



Para acessar uma propriedade ou uma função basta utilizarmos o nome do objeto seguido da propriedade/função separada por ponto. Por exemplo:

```
console.log(carro.modelo)
```

```
carro.acelerar()
```

Nesse caso teremos as saídas:

```
gol
```

```
Acelerando...
```

JS

Array de objetos

Visto como utilizar **arrays** e **objetos**, podemos juntar as duas coisas e criarmos **Arrays de objetos**.

Um array de objetos nada mais é do que uma lista de objetos, ou seja, uma lista que possui itens que possuem propriedades e funções.

Para declararmos um array de objetos, juntamos as duas sintaxes:

```
const productList = [{}]
```

Ao lado temos essa lista populada:

```
const productList = [{
  nome: "Computador",
  marca: "Lenovo",
  preco: 3000
},
{
  nome: "Tablet",
  marca: "Samsung",
  preco: 2000
},
{
  nome: "Celular",
  marca: "Apple",
  preco: 6000,
}
]
```



JS

Percorrendo um array de objetos



Assim como fizemos com o array, podemos percorrer também um array de objetos com a estrutura `forEach`, conforme a seguir:

```
// EXIBINDO ARRAY DE OBJETOS COM FOREACH
productList.forEach(product => {
    console.log(`Produto: ${product.nome}`)
    console.log(`Marca: ${product.marca}`)
    console.log(`Preço: ${product.preco}`)
    console.log()
})
```

Produto: Computador
Marca: Lenovo
Preço: 3000

Produto: Tablet
Marca: Samsung
Preço: 2000

Produto: Celular
Marca: Apple
Preço: 6000

Assim teremos a saída ao lado:

JS



JS

JS Aula 02.2:

Classes

O que são classes em JavaScript?



Em JavaScript, **classes** são uma forma de criar **objetos** e organizar o código de forma mais clara e reutilizável. Elas funcionam como "**modelos**" para criar múltiplos objetos com as mesmas **propriedades** e **métodos**.

As classes foram introduzidas no **ECMAScript 2015 (ES6)** e são basicamente uma **sintaxe especial para funções construtoras**, tornando o código mais limpo e fácil de entender.

```
class Person {  
  constructor() {  
    this.name = 'John Doe';  
  }  
  
  getName() {  
    return this.name;  
  }  
}  
  
const person = new Person();  
  
console.log(person.getName());  
// John Doe
```

JS

Criando uma classe no Javascript



```
// CLASSES NO JAVASCRIPT
class Carro {
  // Nome de classes devem iniciar com a primeira letra maiúscula.
  // ATRIBUTOS
  constructor(marca, modelo, ano) {
    this.marca = marca;
    this.modelo = modelo;
    this.ano = ano;
  }
  // MÉTODOS
  buzinar() {
    return "Beep! Beep!";
  }
}
```

JS

Criando instâncias (objetos)



```
// Criando uma instância (objeto) da classe Carro
const carroPopular = new Carro("Fiat", "Uno", "2012");
document.write(
  `

0 carro ${carroPopular.marca} modelo ${carroPopular.modelo} é do ano
  ${carroPopular.ano} e faz ${carroPopular.buzinar()}</p>`
);

// Instância carroEsportivo
const carroEsportivo = new Carro();
carroEsportivo.marca = "Chevrolet";
carroEsportivo.modelo = "Camaro";
carroEsportivo.ano = "2024";
document.write(
  `

0 carro ${carroEsportivo.marca} modelo ${carroEsportivo.modelo} é do ano
  ${carroEsportivo.ano} e também faz ${carroEsportivo.buzinar()}</p>`
);


```

JS

Adicionando novos métodos e atributos



```
// Adicionando um novo atributo  
carroEsportivo.corNeon = "Azul";
```

```
// Adicionando um novo método  
carroEsportivo.turbo = function () {  
    return "Vrummmm! O carro está acelerando!!!";  
};
```

```
document.write(  
    `

O carro ${carroEsportivo.marca} ${carroEsportivo.modelo}  
também tem neon da cor ${carroEsportivo.corNeon}.  
${carroEsportivo.turbo()}</p>`  
);


```

JS